Name: Parabjot Chander, Ryan Mohamed, Ching Kung Lin

CSCI 348 Data Communications

Group ID: 4

Petmatcher

Professor Mahbubur Rahman

December 9, 2022

Project Report

Work Distribution: Ryan Mohamed: 40%, Ching Kung Lin: 30%, Parabjot Chander 30%

Development stack: MySQL, Node.js (Express & Socket.io [auth server & socket server]), React.js

The fully deployed and working web app can be found at: https://petmatcher.netlify.app

The backend source code can be found at: https://github.com/ryanmohamed/pandagum-client

The front-end source code can be found at: https://github.com/ryanmohamed/pandagum-server

Project Initial Idea and Description:

The app began as an idea to be an online two-player pet generator intended for couples and families looking for the ideal pet. Users connect to the server and establish a socket for this connection. Upon connection, the site asks users if they'd like to enter a pool for random pair ups between users, or if they want to enter the specific room using a 4 digit room id. Upon entering a room through a room id, an enumerated channel will be created in which sockets can connect or disconnect. For these purposes, the number of connected sockets in one channel is limited to 2

clients/users. Otherwise, there will potentially be many random users connecting to one single channel.

The server should incorporate a heartbeat mechanism that pairs up random users every few seconds by creating an untaken random channel. The system will be a client-server architecture mimicking a P2P connection, where the server acts as the middleman for the two sockets (clients/users) in one 1 room to communicate and share data. Once two users are connected via socket to the same channel or room, the app will display a user interface with a question field, form field, and chatroom side panel. The server sends questions out its transport layer to both users, where they respond back with their desired keywords and completion status with their sockets.

The server should implement a timer for the two clients to respond back for the current question. If they don’t respond within a given timeframe the server should randomly choose their answer and move on. The two users will type in as many keywords as they want, and the chat room will be available for two players to communicate with each other and discuss ideas, thoughts, etc. (until the timer runs out).

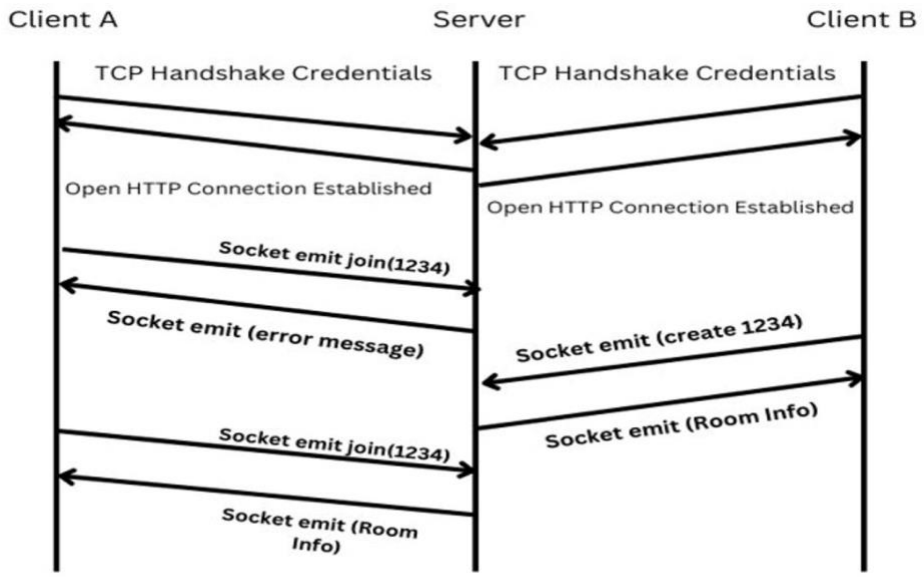
Project Chronological Order (Plain English):

Once a user has navigated to the web app's domain name (found at the top of paper), they can either click the "try it now" or "login in" button to redirect them to the login/signup route of the page. The user can either enter an existing account with email and password, or create a new account and sign in in one step by providing a username, email and password.

Once logged in (with credentials), one user, let's call him or her "User A," has to create the room. Examples of making a room can be by typing or creating Room 1234. "User B" will

join the room as "User A" will be waiting for another user. In User A's screen, it will start looking for a match. Once both users have joined the room, the room is ready and they will be given multiple choices and short answer questions related to pets. While answering the questions, they can message one another (chat feature). Once both users have finished answering their questions, the server will pack their answers into one payload, eliminating duplicate answers and perform an image search for the joined words (would be replaced with AI input). Once the server has retrieved an image from its image search, the server will send the clients (Users A and B) their respective images, and the users will be able to see their ideal pet rendered on their device.

Acknowledgment Diagrams and Implementation Details Explanation:



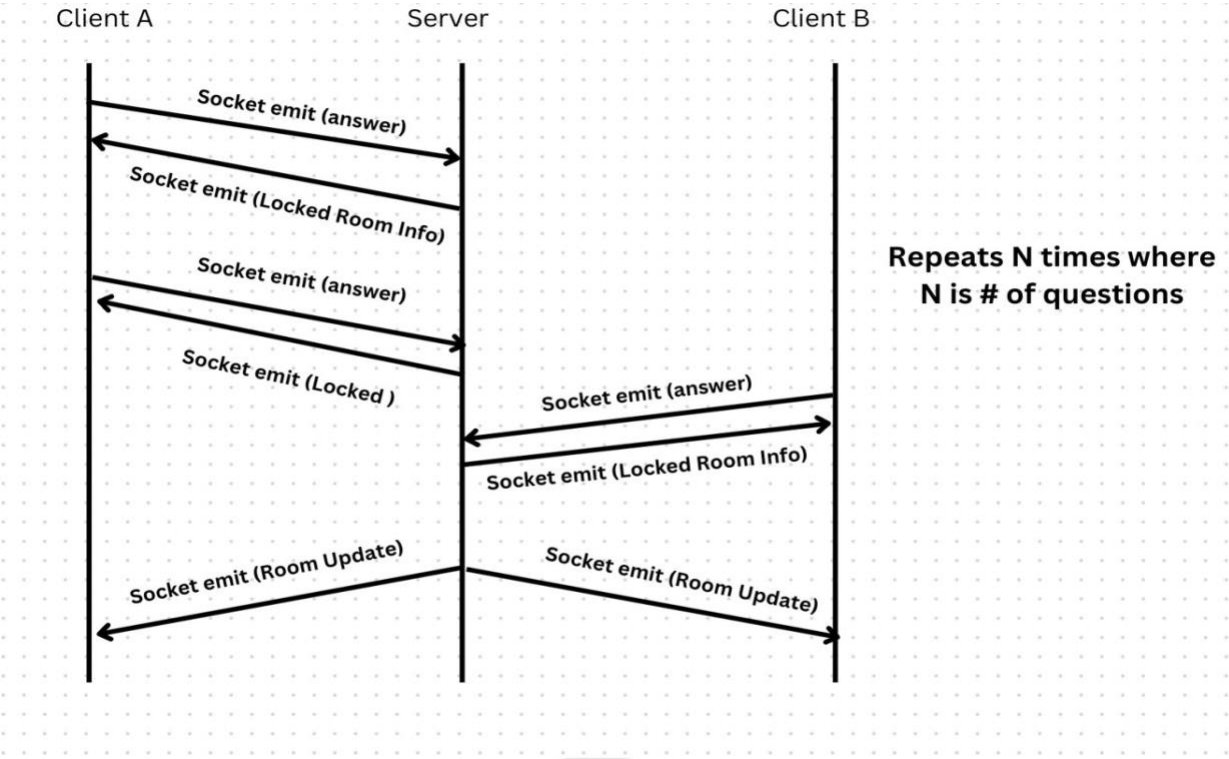
As you can see in the diagram, when a client wants to form a connection through socket with the socket server, a TCP handshake must be performed. This TCP handshake requires some kind of credentials from the client. Whilst on the login page, the data entered in the potential

username, email and password forms are sent in an HTTP POST request to an authentication server hosted separately. The authentication server hashes these values using a secret key. The authentication server then makes a HTTP GET request to a MySQL database server, to verify the user exists or can exist in the database. The authentication server then uses the RefreshToken potentially returned from the database to generate an AccessToken to be used as credentials in connections with the main socket server. (RefreshTokens and AccessToken are related to security and will not be explained in this paper) The authentication server then sends the AccessToken as an HTTP response with the status code 200, to the client, indicating the logged in or signed up successfully. The client securely stores its credentials in its browser cookies, and implicitly passes it in every request to the socket server. The main reasoning for the above is that before any potential client can form a socket that allows it to send data from its application layer through its transport layer to the server, credentials are required to ensure malicious connections aren't being made. The socket server uses middleware to authenticate the tokens it receives. If it receives an invalid token it will reject the incoming or already established connection.

Because credentials are required for the socket to be established, the origin of the incoming request must be known to the socket server. So it is pre-established in setup that all requests are made from the front-end domain name (petmatcher.netlify.app) [CORS issue].

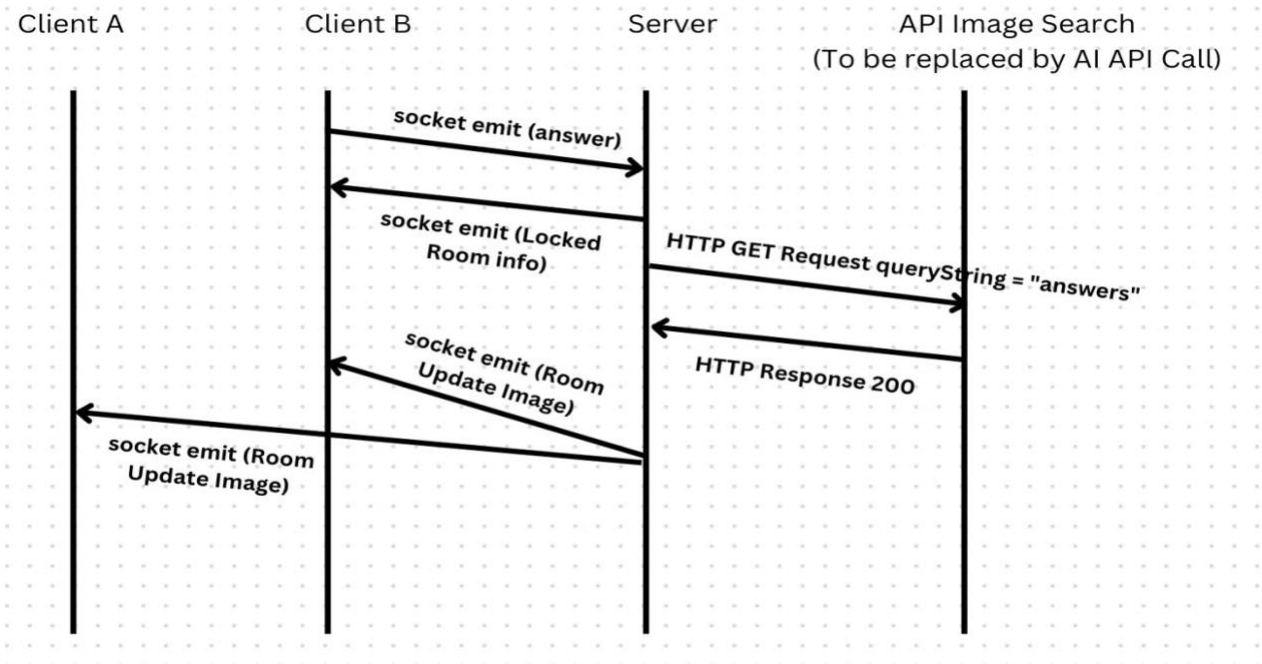
At the point where we have two users logged in, A and B, User A tries to join Room 1234. We can see that the server emits an error message. The socket server must check if the room exists before allowing a user to form a socket connection with the arbitrary channel. It means that one User has to create the room before others can join it. We can't have a user join a room first. In the diagram, User B creates the room where the server approves and gives additional information to User B, such as a mostly empty dictionary representing the state of the

waiting room. Now User A attempts to join the room and is successful. The server then adds a ready status to the dictionary state of the room and sends the data to each client in the room for their client-side to render appropriately.



Synchronization of the answering of questions in a room was a challenge to be considered.The workflow implemented can be found in the above diagram. Client A answers a question, and the server appends a lock status to the dictionary state of the room for User1, meaning that Client A can’t answer another question since its lock is set to false (blocking). Now the only client in the room that can provide an answer to a question is the remaining Client B (unblocked). Once Client B answers a question, Client B is also locked in the dictionary state of the room. When both Client A and B are locked on the socket server end, the room is reupdated by setting both locking statuses to false and the question is incremented, new questions pop up for both clients that are rendered appropriately, and both clients can answer questions again. The same cycle appears for n times, n being the number of questions.

In this diagram, Client B answers the nth question, and the number of questions left in the state of the room is 0. After every question is answered by both parties, the server checks if there are no questions remaining. If so the server emits a match-over event to the clients in the room with an updated room state, of which they will use to move onto the next screen to wait for an image. The server concatenates each of the answers provided by the users, turns it into a set to eliminate duplicate entries, and saves it as a payload object for later use. The server then asynchronously makes a HTTP GET request to the Image Search API (in this case Bing Image Search), providing a query string containing the payload saved before. If successful, the Bing Image Search API responds with a HTTP response with a 200 status, as well as a body of information containing the URLs for all the retrieved images. The server selects the first search result and updates the state of the room with it, and finally emits a room state update to each of the clients in the particular room. The client-side then uses the image URL in the room state to render the image for each user. Some edge cases had to be considered if users prematurely exit a room. The socket adequately handles each “leave” event in any scenario. In any event of a client leaving a particular room that may be in progress, the socket server will adequately remove all references to the leaving member and reinitialize the dictionary representing the game state. It then emits a room state update event to any client left in the room. If both clients leave the room, it is discarded, and its dictionary object is removed from the collection on the server.



Message Passing Between Clients (Chat Feature) Explanation/Pseudocode:

Socket.io provided great ease of use when creating the chat feature. Because of each socket's member function "to", we could specify specific sockets to send message events (paired with message data). Because the chat is not a common feature through all phases of the web app, it is specific to the 2 users in a room. Messaging can only take place between 1 user and 1 other specific user. The client simply emits message data to the server using an allocated chat event, where the server appropriately "broadcasts" to all other sockets in that room. A broadcast is performed so the data is duplicated on the sender's end. Since the number of users in one room is limited to 2, the message will only ever be sent from Client A to Server to Client B (or the inverse).

Server Side:

Step 1: Create a Server and pass a port number as a parameter so that server will be listening on.

Step 2: Every time a user logins and creates/joins a room, he or she is given a socket id.

Step 3: When the client hits the send button with a message, the server receives the message through a socket.on arrow function, the server is actively on and listening for any data (messages) that might be sent to it from one of the clients.

Step 4:

- The server sends the message to the other client through an append message function inside the function.
- A message element object is created and is set equal to the message input the client sent.
- The server adds the message element to a message container which is visible to the two clients.

The message container is visible in the user interface for the web page. This method is called inside the socket.on arrow function in step 3. That means when a client inputs a message, the other client can see the message because the server received the message and sent that message to a message container which is visible in the GUI.

Client Side:

Step 1: The Client retrieves its socket variable by creating a variable and setting it equal to io('http://localhost:3000'). **In this case, the port is 3000. But in our final product, it wasn't the port the server was running on, not on localhost. It is just written for pseudocode purposes.**

Step 2: When the user types something onto the chat and hits the send button, an event listener method is called, meaning that the message the client inputted is sent to the server by socket.emit('send-chat-message', message). Finally, the message input is set to an empty string so the client can send more messages.

References and Information:

Client-side JWT authentication and use in HTTP requests adapted and inspired by:

https://www.youtube.com/watch?v=27KeYk-5vJw&list=LL&index=61&t=474s&ab_channel=DaveGray

All back-end socket work was managed using:

https://socket.io/docs/v4/